Chapter 2: Time Series Regression Models

Honours Time Series Module

September 2026

---

1 Introduction

This chapter will deal with situations where we have explanatory variables that we would like to include in our time series models. Explanatory (predictor / input) variables will often allow us to construct better models and to make better predictions. We will include seasonal terms and trends (change in mean over time) under this concept of explanatory variables. A regression approach also allows us to ask interesting questions about, for example, interventions, and to adapt models for non-normal data such as counts or binary data (e.g., using a GLM approach).

The emphasis is on *modelling* the system: rather than removing trends, seasonal effects, etc. until we have a stationary process, we include these terms explicitly in our models.

Regression-type models for time series data are important in many fields: medicine, econometrics, engineering, environment, etc.. One consequence of so many fields being involved in model development is that all have added their own terminology. Consequently this area of time series (regression models for time series data) can be very confusing, and I will start with an overview of the main types of regression models for time series data.

2 Overview: Types of Models

2.1 ARMA(p, q)

$$y_t = \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} - \theta_1 z_{t-1} - \cdots - \theta_q z_{t-q} + z_t$$

z_t is a white noise process. There is a single time series, no explanatory variables. These are the models you will have encountered so far.

2.2 ARMAX(p, q)

$$y_t = \beta x_t + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} - \theta_1 z_{t-1} - \cdots - \theta_q z_{t-q} + z_t$$

There is a single response variable. Explanatory variables are added (on the right hand side) to an ARMA(p, q) model.

2.3 Regression with ARMA errors

$$\begin{aligned} y_t &= \alpha + \beta x_t + n_t \\ n_t &= \phi_1 n_{t-1} + \cdots + \phi_p n_{t-p} - \theta_1 z_{t-1} - \cdots - \theta_q z_{t-q} + z_t \end{aligned}$$

The ARMA-errors model does not include auto-regressive terms of the response variable, but it can include lagged versions of the explanatory variables. We will look at these models more closely in this chapter.

Typically, all of the ARIMA-type models above are fitted using a state-space approach, and assuming normal errors.

These models can be generalized to non-normal data: GARMA.

2.4 Transfer Function Models

The terminology of **transfer function** comes from engineering, and refers to how the effect of the input variable is transferred to the output variable. In the simplest case it could just be through a linear relationship expressed through a regression coefficient. But it gets more complicated with autoregressive terms in the model. We will not deal with transfer function models directly, only with some special cases mentioned above. These models include both, regression with ARMA errors and the ARMAX model, as special cases.

2.5 Dynamic Regression Models

Dynamic regression models include lagged variables to incorporate feedback over time, as opposed to *static* models which only include current events.

We can include lagged explanatory /exogenous variables (distributed lag models), lagged response /endogenous variables (AR models), or lagged error terms / innovations (MA models), and different combinations of these. These models overlap with the types of models named above (regression with ARMA errors, ARMAX and transfer function models, distributed lag models).

The terminology of 'dynamic regression model' overlaps with **dynamic linear model**, which usually refers to a state-space model, where the state variables / parameters are allowed to change over time, and we will mean these state-space models when we refer to dynamic models.

2.6 Distributed Lag Models

These are regression models in which the response is modelled in terms of explanatory variables and their lags. This can be problematic, because lagged versions of the same variable may be highly correlated, introducing multicollinearity. There are various approaches to restrict the coefficients in order to reduce this problem, however, we will not deal with these here. These models may also be called *lagged regression models*, and they are a special case of transfer function models (if errors follow an ARMA process).

2.7 Generalized Least Squares

GLS is an approach we can use to fit regression models with ARMA errors. It is not directly a type of model.

2.8 Vector Autoregressive Models (VAR)

This is an approach for multiple time series (multivariate time series analysis) in which two or more time series are modelled as interrelated. This is an extension of AR(p) models to a multivariate time series. Explanatory variables can be added. VAR models investigate the effects of one variable on others, and are often used for *Granger causality*.

3 Useful Predictor Variables

In this chapter the emphasis is on how to incorporate explanatory variables into our time series models, so that we can better understand how they affect or change the response. Here is a summary of a few types of explanatory variables especially useful in time series analyses (Hyndman 2021, sec. 7.4).

1. **Linear trend:** To add a linear trend term we could include an index variable $t = 1, ..., T$. Instead of a linear trend we can have a more flexible trend model by using quadratic or cubic terms in t, or by using splines.

2. **Dummy variables** can be used to investigate the effects of public holidays, weekends, seasons, outliers or special events.

Splines are flexible (semi-parametric) curves that can be used to model non-linear relationships.

3. **Interventions:** We may want to model the effect of an intervention, such as a new policy, on the response. The effect can either be immediate and last only for one observation, i.e. the response might have a spike. To model this, we would include a dummy variable with 1 for the time of the spike and 0 elsewhere.

Other interventions might result in a change in level, so that we would expect a step change in the mean. This can be modelled with a step function by including a variable with 0 before the change and 1 for time points $\geq$ the time point of the step.

A change in slope can be handled with an interaction between a step variable (x here) and a linear trend variable (t here):

$$y_t = \beta_0 + \beta_1 t + \beta_2 t * x + e_t$$

where β_2 is the change in slope. This will result in a piece-wise linear trend (continuous at the change point).

If we suspect exponential decay of the intervention effect, we could add a variable which decreases at a certain rate (rate parameter can be estimated as well).

4. **Lagged variables (distributed lags):** Some explanatory variables may have a delayed effect on the response, or may affect the response for several time steps into the future. For example, advertising may affect sales for the next few weeks or months; high temperatures may affect mortality for the next few days. To account for such lagged effects we can add lagged versions of the explanatory variables (distributed lags). Because this introduces multi-collinearity, several approaches to restrict parameters are available (e.g. exponential decay).

5. **Harmonic Regression:** To model seasonal effects (regular periodic patterns), sine and cosine terms can be added to regression models.

$$y_t = \beta_0 + a \sin(2\pi\omega t) + b \cos(2\pi\omega t)$$

ω here is the frequency: number of cycles per unit time. Mostly this frequency will be a fraction, e.g. for monthly data, if we expect one cycle per year = 12 months, the frequency = $\frac{1}{12}$. 2π is a full cycle in radians, $2\pi\omega t$ then measures where in the cycle we are at time t (in radians). Sometimes these models are defined in angular frequency f, which measures radians completed per unit time. For the monthly data this angular frequency would then be $\frac{2\pi}{12}$ and the above model would be written as

Actually, what we would like to fit is a term such as $A\sin(2\pi\omega t + \phi)$ with amplitude A and phase shift ϕ. This model is not linear but can be rewritten: $A\sin(2\pi\omega t + \phi) = asin(2\pi\omega t) + bcos(2\pi\omega t)$, which now is linear in parameters if ω is known.

$$
\begin{aligned}
y_t &= \beta_0 + a\sin(ft) + b\cos(ft) \\
&= \beta_0 + a\sin\left(\frac{2\pi t}{12}\right) + b\cos\left(\frac{2\pi t}{12}\right)
\end{aligned}
$$

The above model is linear if ω is known, and the parameters can be estimated by fitting a linear regression model. If the frequency is not known, this can sometimes be estimated from a periodogram or as a parameter in a model.

Wavelength (or period) is the reciprocal of frequency: number of time units in one cycle = length of cycle in time units.

Fourier Terms: Jean-Baptiste Fourier was a French mathematician, born in the 1700s, who showed that a series of sine and cosine terms of the right frequencies can approximate any periodic function. Using this idea, we can approximate any complicated cyclical function (or seasonal pattern) by adding Fourier terms as follows:

$$
y_t = \sum_{j=0}^{n/2} \left(sin(2\pi j\omega t) + b_j cos(2\pi j\omega t)\right)
$$

The Fourier frequencies are $\omega_j = \frac{j}{m}, \qquad j = 0, 1, 2, \ldots, \frac{m}{2}$, where m is the full wavelength / number of time units in a full cycle / season (e.g. 12 months in a year). As j increases, the frequency of the individual terms increases, shorter cycles.

In practice we use only a few sine and cosine terms to approximate our seasonal function. The following code shows how one would create the first three sine terms to add to such a regression model (`x1`, `x2` and `x3` would be added to the model as explanatory variables, in addition to the corresponding cosine terms).

The Fourier terms form an orthogonal basis, i.e. all terms are independent of each other.

```
ti <- 1:100
x1 <- sin(2 * pi * ti / 100)
x2 <- sin(2 * pi * ti * 2 / 100)
x3 <- sin(2 * pi * ti * 3 / 100)
```

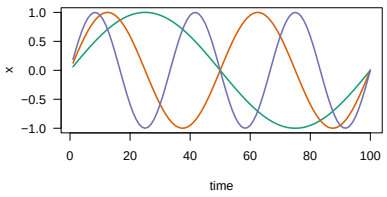


Figure 1: First three sine terms with wavelengths 100, 50, 100/3.

3.1 Example: Monthly Sea Surface Temperature

If the frequencies of interest are known, it is easy to add the sine and cosine terms in a linear regression model. Here is an example on monthly sea surface temperature data (`nino3`, R library `tseries`) (`nino3` is for region 3, an area in the Pacific), fitting a yearly cycle (1 cycle per 12 months). The yearly cycle is clear in the periodogram.

```
spectrum(nino3, spans = 4, main = "")
```

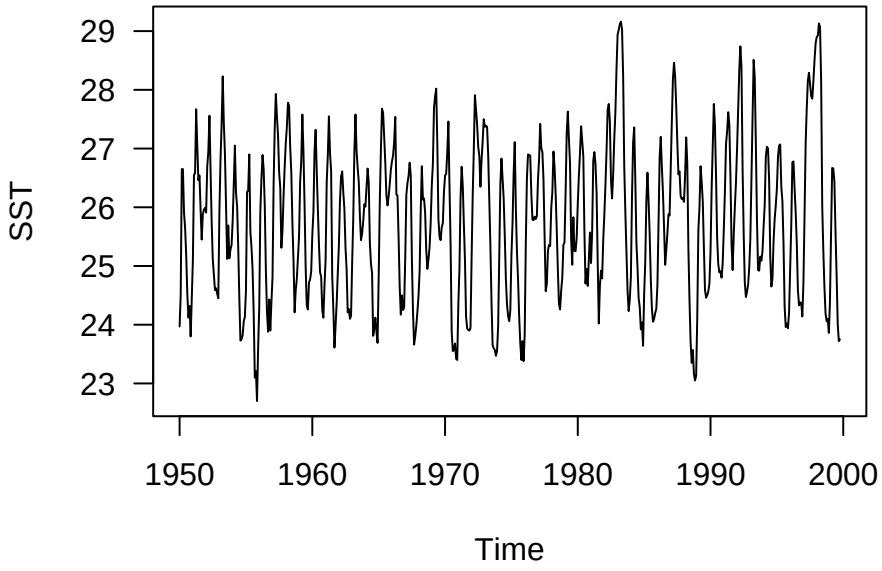


Figure 2: Monthly sea-surface temperature, periodogram and fitted values from a regression model with harmonic terms.

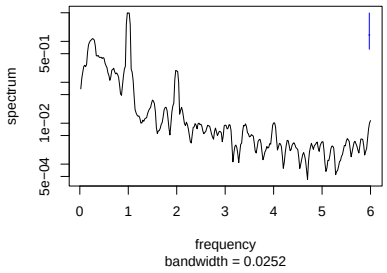


Figure 3: Periodogram for monthly sea-surface temperature.

Note the sin and cos terms in the `lm` model.

```
y <- as.numeric(nino3)
ti <- 1:length(y)

m.harm <- lm(nino3 ~ sin(2 * pi * ti / 12) +
                     cos(2 * pi * ti / 12))

p.harm <- predict(m.harm)
plot(nino3, las = 1)
lines(as.numeric(time(nino3)), p.harm, col = cols[2])
```

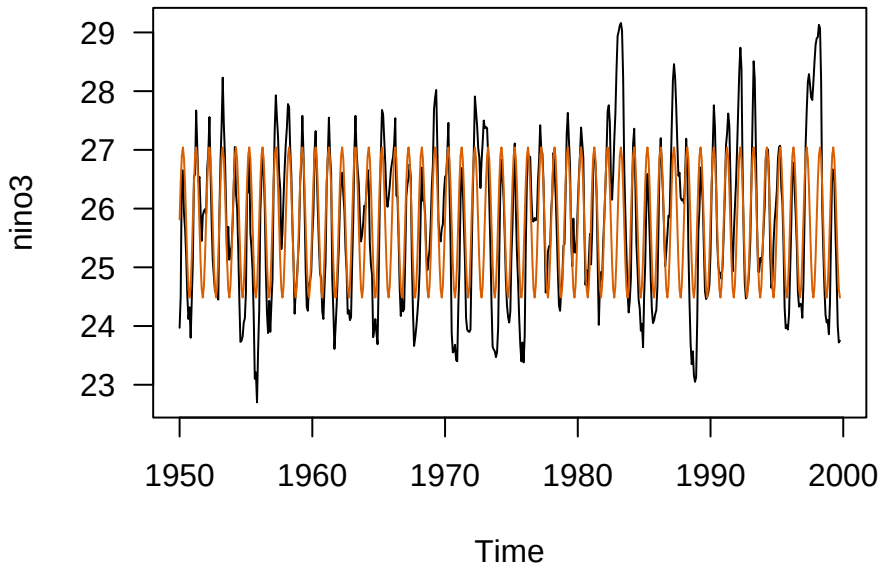


Figure 4: Fitted values from a regression model with harmonic terms fitted to the nino3 data.

An El-Nino event is when the sea surface temperate is warmer than usual for at least 6 consecutive months. After adding the average seasonal pattern we have some idea of what is 'usual'

There appears to be a lower frequency component (see spectogram) in the time series as well, and the amplitudes of the seasonal cycles are much more variable than captured by this simple model, but this is a good start for modelling the seasonality in sea surface temperature. Harmonic regression models assume that the seasonal effects remain constant / don't change over time.

3.2 Example: Magnitude of a variable star

These data are magnitudes (measure of brightness) of a variable star (T. Ursa Major) measured at midnight on 600 successive days. This series can be described almost completely in terms of two trigonometric functions with periods 24 and 29 days (`star` in R package `TSA`).

```
library(TSA)
data(star)
par(mfrow = c(1, 2))
plot.ts(star, las = 1, ylab = "magnitude")

spec <- spectrum(star, spans = 2, plot = FALSE)
plot(spec, main = "")

y <- star
ti <- 1:length(star)
```

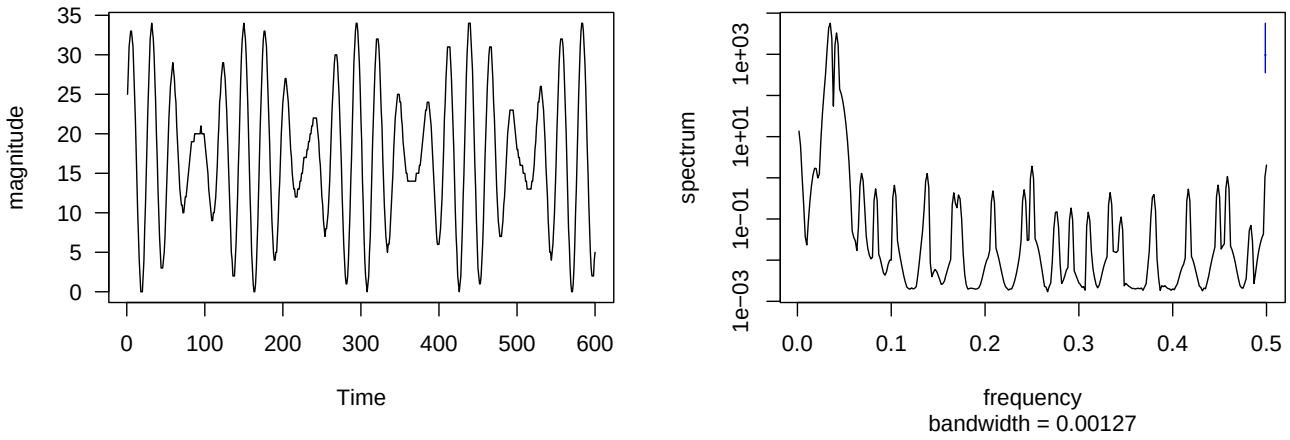


Figure 5: Magnitudes of a variable star measured on 600 consecutive days, and periodogram.

In the following code maximum likelihood is used to estimate the important frequencies, at the same time as fitting the harmonic regression model. First, we define the log-likelihood (returns the negative log-likelihood).

`exp(p[4])` is a trick to ensure that the estimated frequency will be positive.

```r
ll.harm4 <- function(p, yi, ti) {
  b0 <- p[1]
  b1 <- p[2]
  b2 <- p[3]
  f1 <- exp(p[4])
  f2 <- exp(p[5])
  b3 <- p[6]
  b4 <- p[7]
  sigma <- exp(p[8])
  mu <- b0 + b1 * sin(2 * pi * ti * f1) +
             b2 * cos(2 * pi * ti * f1) +
             b3 * sin(2 * pi * ti * f2) +
             b4 * cos(2 * pi * ti * f2)

  ll <- sum(dnorm(yi, mean = mu, sd = sigma, log = TRUE))
  return(-ll)          # return negative log-likelihood
}
```

```r
p.init = c(17, 3, -3, log(1/24), log(1/29), 5, 1, 1)

out.optim <- optim(par = p.init, fn = ll.harm4, yi = y, ti = ti)
out.optim
```

```
$par
[1] 17.6495912  5.2811179 -5.3027064 -3.1728928 -3.3630315  7.8562929  2.0302685
[8]  0.9013771

$value
[1] 1389.671

$counts
function gradient
     501       NA

$convergence
[1] 1

$message
NULL
```

```r
p <- out.optim$par
pred2 <- p[1] + p[2] * sin(2 * pi * ti * exp(p[4])) +
                p[3] * cos(2 * pi * ti * exp(p[4])) +
                p[6] * sin(2 * pi * ti * exp(p[5])) +
                p[7] * cos(2 * pi * ti * exp(p[5]))

plot.ts(star, las = 1, ylab = "brightness")
lines(pred2, col = "blue")
```

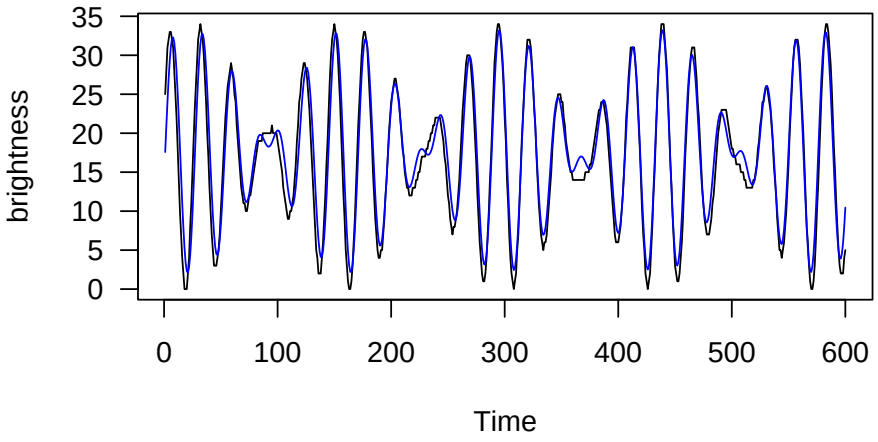


Figure 6: Observed brightness of variable star Ursus Major (black) and estimated brigthness using a model with sine and cosine terms for two different frequencies (estimated) (blue).

Parameter estimates 4 and 5 estimate the frequencies (on log scale). -3.173 translates into a frequency of 0.042 cycles per day = one cycle every 23.9 days. -3.363 translates into a frequency of 0.035 cycles per day = one cycle every 28.9 days. Note that some time series can be modelled well without the need for any error autocorrelation structure, i.e. the explanatory variables (which are auto-correlated) may explain enough of the autocorrelation structure in the response.

4 ARMAX and ARMA Errors

This section is about combining time series aspects (autocorrelation) with regression models (predictor variables). Regression models are easy to interpret, and there is well-established theory and software for prediction, model comparison, model checking, etc. It makes sense to develop regression-type models that can deal with time series data (autocorrelation).

In regression models we assume independent, equally distributed errors. If we generalize to generalized linear models (which are also a type of regression model) we still have independent errors, although they are no longer equally distributed.

In the presence of serial correlation, OLS standard errors can be very wrong, OLS estimates are no longer efficient, although they are still unbiased and consistent.

If we add lagged *response* variables, OLS assumptions are further violated, in that the errors and predictors are no longer independent, and regression coefficients become biased (the beta coefficients absorb some of the error effects). This situation becomes even worse when the error terms are auto-correlated (MathWorks).

Efficient: minimum variance.
Consistent: converges to true value with increasing sample size.

In this section we will briefly look at regression-type models which allow for autocorrelation in the data, through either including lagged

predictors (in response or explanatory variables), or through allowing dependence in the error terms.

Although adding explanatory (predictor) variables to ARMA models will often account for a large proportion of the auto-correlation in the errors, auto-correlated errors often remain a problem. With auto-correlated errors, standard errors, prediction intervals, tests, etc. based on linear regression theory are not valid (because this theory relies on independent errors).

In the Econometrics literature response and explanatory variables are often referred to as *endogenous* and *exogenous* variables, respectively.

4.1 Example

As an illustration we start with a simple example, with one explanatory variable x. The following code simulates data from model

$$y_t = 1 - 4x + \eta_t, \qquad \eta_t = \phi\eta_{t-1} + e_t, \qquad e_t \sim N(0, \sigma^2).$$

```
library(astsa)

set.seed(12)
eta <- numeric(100)

eta[1] = 10
x <- rnorm(100, 5, 1)
phi <- 0.95

## auto-correlated errors
for (i in 2:100) {
 eta[i] <- phi * eta[i-1] + rnorm(1, 0, 1)
}

## response, linear relationship + auto-correlated errors
y <- 1 - 4 * x  + eta
```

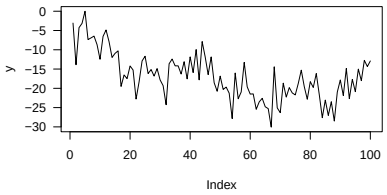


Figure 7: Simulated data from Example 1: simulated with predictor and AR(1) errors.

After fitting a simple linear regression model, we find that the residuals are auto-correlated (AR1, established from ACF and PACF plots).

```
Call:
lm(formula = y ~ x)

Residuals:
    Min      1Q  Median      3Q     Max
-8.4110 -3.2916  0.0579  2.6931 11.2852

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)    8.637      2.497    3.46 0.000803
```

```
x             -5.140      0.495  -10.38  < 2e-16

Residual standard error: 4.261 on 98 degrees of freedom
Multiple R-squared:  0.5238,    Adjusted R-squared:  0.5189
F-statistic: 107.8 on 1 and 98 DF,  p-value: < 2.2e-16
```

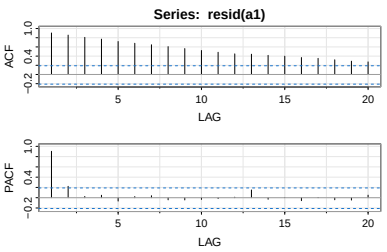


Figure 8: Autocorrelation plots for residuals from OLS regression for simulated data from Example 1.

One approach to account for the autocorrelation in the residuals is to add an AR(1) term:

$$y_t = \beta_0 + \beta_1 x_t + \phi y_{t-1} + e_t$$

This is an ARMAX(1, 0, 0) model, which can be fitted in R using the `dynlm` function (library `dynlm`) or just a linear regression model

Much of the research into lagged response variables and dynamic models seems to be from the political sciences (e.g. Wilkins 2018).

```
options(show.signif.stars = FALSE)
library(dynlm)
y <- ts(y)
summary(arx1 <- dynlm(y ~ x + L(y, 1)))
```

```
Time series regression with "ts" data:
Start = 2, End = 100

Call:
dynlm(formula = y ~ x + L(y, 1))

Residuals:
    Min      1Q  Median      3Q     Max
-5.8881 -1.7148  0.1317  1.5929  6.8288

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 14.87839    1.56945    9.48 1.95e-15
x           -4.50627    0.29577  -15.24  < 2e-16
L(y, 1)      0.55852    0.04114   13.58  < 2e-16

Residual standard error: 2.489 on 96 degrees of freedom
Multiple R-squared:  0.8321,    Adjusted R-squared:  0.8286
F-statistic: 237.9 on 2 and 96 DF,  p-value: < 2.2e-16
```

```
confint(arx1)
```

```
                2.5 %     97.5 %
(Intercept) 11.7630540 17.9937314
x           -5.0933681 -3.9191720
L(y, 1)      0.4768671  0.6401788
```

```
library(Hmisc)
lm.ar1 <- lm(y ~ Lag(y) + x)   # equivalent to arx1
```

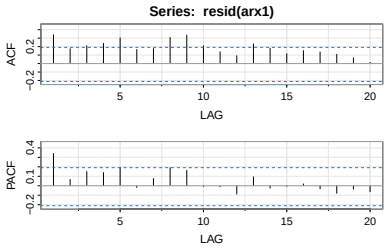


Figure 9: ACF and PACF plotss for model including lagged response variable (arx1).

The autocorrelation in the residuals has decreased but is still there.

When adding lagged response variables to the RHS of a time series, and using OLS to estimate the parameters of such a model can result in biased estimates of the beta coefficient (for x). In the ARMAX model, β_1 (the regression coefficient for x) does not measure the change in mean response per unit change in x, as we would expect from a regression model. Instead, β is the effect of a one unit increase in x on some function of the *difference* between the current and the lagged value of the response, i.e. interpretation of the regression coefficients is not straightforward (Keele and Kelly 2006; The Mathworks Inc., n.d.). However, more recent research suggests that omitting lagged response variables can also introduce bias due to omitted explanatory variables. And this research suggests that the right approach depends very much on the data generating process. So, it is important to think about which variables influence the response, and how.

A note on lags in R: the `lag` function (library `stats`) requires a time series object as input. To obtain x_{t-1} we need to specify `lag(x, -1)`, which is different to most other software packages. An alternative is the `Lag` function in library `Hmisc`: `Lag(x, shift = 1)`.

```
Time series regression with "ts" data:
Start = 2, End = 100

Call:
dynlm(formula = y ~ x + L(y, 1))

Residuals:
    Min      1Q  Median      3Q     Max
-5.8881 -1.7148  0.1317  1.5929  6.8288

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 14.87839    1.56945    9.48 1.95e-15 ***
x           -4.50627    0.29577  -15.24  < 2e-16 ***
L(y, 1)      0.55852    0.04114   13.58  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.489 on 96 degrees of freedom
Multiple R-squared:  0.8321,    Adjusted R-squared:  0.8286
F-statistic: 237.9 on 2 and 96 DF,  p-value: < 2.2e-16
```

An alternative approach (to adding lagged response variables) for modelling autocorrelation is to account for the correlation structure in

the errors by modelling the errors as an ARMA process. For example, the following model defines an AR(1) process for the errors.

$$y_t = \beta_0 + \beta_1 x_t + \nu_t$$

$$\nu_t = \phi \nu_{t-1} + e_t$$

e_t is a white noise process, ν_t is an AR(1) process.

This approach has been criticized, e.g. by Mizon (1995), for placing unrealistic restrictions on the parameter estimates, but it is still a very common approach for dealing with autocorrelation. R's `arima` function fits an ARMA-errors model when an explanatory variable (through `xreg`) is added.

```
(a2 <- arima(y, order = c(1, 0, 0), xreg = x))
```

`arima` uses a state-space representation of the ARIMA model to define the likelihood and estimate parameters.

```
Call:
arima(x = y, order = c(1, 0, 0), xreg = x)

Coefficients:
         ar1  intercept     xreg
      0.9762     4.7905  -3.9967
s.e.  0.0188     3.2704   0.0830

sigma^2 estimated as 1.021:  log likelihood = -144.46,  aic = 294.92
```

```
library(astsa)
sarima(y, 1, 0, 0, xreg = x)    ## equivalent, with more output
```

Hyndman suggests that the two approaches (ARMAX and ARMA errors) don't differ too much in terms of forecasting ability (https://robjhyndman.com/hyndsight/arimax/).

Because of all the different models that can be fitted, and because of all the different naming conventions of these models, it is very important to carefully define the model that was fitted.

Starting with the ARMA errors model:

$$y_t = \beta_0 + \beta_1 x_t + \eta_t, \qquad \eta_t = \phi \eta_{t-1} + e_t$$

can be written as

$$y_t = \beta_0 + \beta_1 x_t + \phi \eta_{t-1} + e_t.$$

The model for y_{t-1} is

$$y_{t-1} = \beta_0 + \beta_1 x_{t-1} + \eta_{t-1},$$

from which we can obtain

$$\eta_{t-1} = y_{t-1} - \beta_0 - \beta_1 x_{t-1}$$

and, substituting,

$$y_t = \beta_0 + \beta_1 x_t + \phi y_{t-1} - \phi\beta_0 - \phi\beta_1 x_{t-1} + e_t = \beta_0 + (x_t - \phi x_{t-1})\beta_1 + \phi y_{t-1} + e_t \quad (1)$$

In this latest arrangement of the model, if $\phi = 0$, we are back to a regression model. If $\phi = 1$ we have a random walk in y_t where the *change* in y_t depends on the change in x_t, i.e. as ϕ shifts from 0 to 1, the meaning of β_1 shifts from the effect of x on the response to the effect of a change in x on a change in y. Equation 1 suggests another approach that is often used in regression if the aim is to get rid of auto-correlated residuals: calculate $y_t^* = y_t - \phi y_{t-1}$ and $x_t^* = x_t - \phi x_{t-1}$, and then continue with an ordinary least squares. The problem here is that ϕ is not known and must be approximated.

If we take the estimate from the ACF function at lag 1 (0.9), we get estimates very close to the values used for simulation.

```
Call:
lm(formula = y_star ~ x_star)

Residuals:
     Min       1Q   Median       3Q      Max
-2.15098 -0.79754  0.00111  0.76128  2.28805

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.21353    0.11303   1.889   0.0619 .
x_star      -3.99263    0.08831 -45.212   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.034 on 97 degrees of freedom
  (1 observation deleted due to missingness)
Multiple R-squared:  0.9547,    Adjusted R-squared:  0.9542
F-statistic:  2044 on 1 and 97 DF,  p-value: < 2.2e-16
```

The regression model with y_t^* and x_t^* and the AR1-error model give very similar estimates for ϕ and β_1, and these are close to the values used for the original simulation (the estimates for the intercept differ). BUT, the estimates from the regression model are for change-in-y for change-in-x?! The two models cannot be compared with AIC because the likelihood is calculated for different responses.

We can compare the AR1-error model (`a2`) with the ARMAX model (`arx`) and (almost) with the original regression model (`a1`). The AR1-error model does best, but that is also the model we simulated from. `a1: y \sim x` has no lag term, and therefore one observation more than the models with lag terms.

	df	AIC
a1	3	577.7
arx1	4	466.5
a2	4	296.9

If $0 < \phi < 1$ we can write the model as

$$y_t = \beta_0 + \beta_1 x_t + \phi y_{t-1} - \gamma x_{t-1} + e_t.$$

This is an autoregressive distributed lags model (distributed lags refer to a model with lagged explanatory variables), so that modelling an AR(1) process in the errors results in a similar model structure as the ARMAX model.

The model

$$y_t - y_{t-1} = \alpha + \beta_1(x_t - x_{t-1}) + e_t$$

is important in animal movement modelling, where $y_t - y_{t-1}$ equals velocity = change in position (Hooten MB (2017)).

4.2 Estimation for ARMA Errors Model using GLS

Generalized Least Squares (GLS) is an estimation method for linear regression models with auto-correlated errors:

$$y = Z\beta + e, \qquad e \sim N(0, \sigma^2 V),$$

where V is the correlation matrix. If we know the covariance matrix V, we can pre-multiply each term by $V^{-1/2}$ to obtain

$$V^{-1/2}y = V^{-1/2}Z\beta + V^{-1/2}e$$

resulting in

$$y^* = Z^*\beta + \delta$$

where now δ is white noise: $Cov(\delta) = I$, and we can use OLS. We obtain a weighted estimate of β

$$\begin{aligned} \hat{\beta} &= (Z^{*T}Z^*)^{-1}Z^{*T}y^* && (2)\\ &= (Z^TV^{-1}Z)^{-1}Z^TV^{-1}y && (3)\\ & && (4) \end{aligned}$$

Usually, however, V is unknown. Without any structure in V, there would be too many elements to estimate. But if we assume some structure for the residual correlation we can reduce the extra number of parameters to estimate to a few. For example, if we are willing to assume that the errors follow an AR(1) process, then $V_{ij} = \phi^{|i-j|}$ (we would only need to estimate the extra parameter ϕ). The correlation function for an MA(q) model is (Pinheiro and Bates 2000, 229):

$$h(k,\theta) = \begin{cases} \frac{\theta_k + \theta_1\theta_{k-1} + \ldots + \theta_{k-q}\theta_q}{1+\theta_1^2+\ldots+\theta_q^2}, & k = 1, \ldots, q \\ 0, & k = q+1, q+2, \ldots \end{cases}.$$

The `gls` (generalized least squares) function in package `nlme` estimates the parameters of the correlation matrix if one defines a correlation structure for the errors (see mortality example below).

```
library(nlme)
m.gls <- gls(y ~ x, correlation = corAR1())
m.gls
```

`gls` estimates both regression coefficients and the correlation parameters by first finding the profile likelihood for the correlation parameters, and finding the corresponding MLEs, and then substituting these MLEs into the OLS equations above to estimate the regression coefficients (method = ML) (Pinheiro and Bates 2000, pg. 202).

Feasible Generalized Least Squares proceeds as follows:

- fit an OLS regression model
- estimate the covariance matrix V from the residuals
- fit the model again using GLS
- iterate until the estimates converge

4.3 Estimation for ARMA Errors Model - minimize independent SSE

Parameter estimates can also be found by minimizing the sum of squares of the **independent** errors. For example, for a regression model with AR(1) errors

$$y_t = \beta x_t + e_t, \qquad e_t = \gamma e_{t-1} + \lambda_t, \qquad \lambda_t \sim N(0, \sigma^2)$$

the independent errors can be written as

$$\lambda_t = (y_t - \beta x_t) - \gamma(y_{t-1} - \beta x_{t-1}).$$

4.4 Example: Mortality, Temperature and Pollution

This example is from Shumway and Stoffer (2011) (Section 5.6 pg. 293 in 3rd edition, Section 3.8 in 4th edition). We want to relate weekly cardiovascular mortality to temperature and particulate levels (Los Angeles, 1970 - 1979, data source: `lap` in library `astsa`).

We start with a linear regression model, and use this to get some idea of the autocorrelation structure. The plot of mortality against temperature suggest a non-linear relationship, perhaps quadratic, so we add a quadratic term for temperature, and a linear trend term (`ttt`), because it looks like mortality decreased over time.

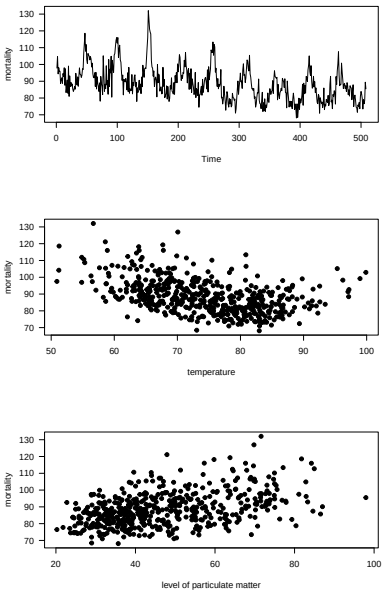


Figure 10: Cardiovascular mortality plotted against time, temperature and particulate levels.

```
ttt <- 1:508     # time variable
dim(lap)

m1 <- lm(cmort ~ ttt + tempr + I(tempr^2) + part )
summary(m1)

m1.res <- resid(m1)
```

Based on the ACF and PACF plots of the residuals, an AR(2) model is suggested. Therefore, we next fit the following model

$$M_t = \beta_1 + \beta_2 t + \beta_3 T_t + \beta_4 T_t^2 + \beta_5 P_t + \eta_t \tag{5}$$

where η_t is AR(2)

$$\eta_2 = \phi_1 \eta_{t-1} + \phi_2 \eta_{t-2} + e_t$$

and e_t is white noise.

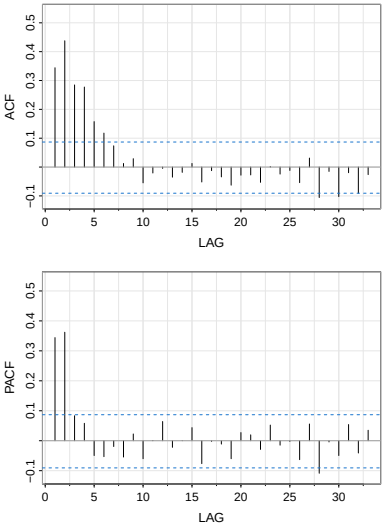


Figure 11: ACF and PACF for mortality regression model residuals.

```
cmort <- lap[,3]
trend <- time(cmort) - 1970
temp <- lap[, 4]
temp <- temp - mean(temp)
part <- lap[, 11]
temp2 <- temp^2

(mort_ar2 <- arima(cmort, order = c(2, 0, 0),
            xreg = cbind(trend, temp, temp2, part)))
```

```
Call:
arima(x = cmort, order = c(2, 0, 0), xreg = cbind(trend, temp, temp2, part))
```

```
Coefficients:
         ar1     ar2  intercept    trend     temp   temp2    part
      0.3848  0.4326    30.1810  -0.0292  -0.0190  0.0154  0.1545
s.e.  0.0436  0.0400    14.0094   0.0081   0.0495  0.0020  0.0272

sigma^2 estimated as 26.01:  log likelihood = -1549.04,  aic = 3112.07
```

We can also fit model Equation 5 using generalized least squares:

```
library(nlme)

g1 <- gls(cmort ~ trend + temp + I(temp^2) + part,
          correlation = corARMA(p=2))
summary(g1)
```

```
Generalized least squares fit by REML
  Model: cmort ~ trend + temp + I(temp^2) + part
  Data: NULL
       AIC      BIC    logLik
  3140.488 3174.253 -1562.244

Correlation Structure: ARMA(2,0)
 Formula: ~1
 Parameter estimate(s):
     Phi1      Phi2
0.3939043 0.4381177

Coefficients:
                Value Std.Error   t-value p-value
(Intercept) 30.374121 15.233863  1.993855  0.0467
trend       -0.029182  0.008841 -3.300541  0.0010
temp        -0.009702  0.043220 -0.224484  0.8225
I(temp^2)    0.015370  0.002021  7.606690  0.0000
part         0.150143  0.024912  6.026867  0.0000

 Correlation:
          (Intr) trend  temp   I(t^2)
trend      0.993
temp       0.017 -0.031
I(temp^2) -0.005  0.009 -0.080
part      -0.046  0.033 -0.602  0.037

Standardized residuals:
       Min         Q1        Med         Q3        Max
-2.1033378 -0.6999939 -0.1497608  0.5200921  4.1160089

Residual standard error: 7.996799
Degrees of freedom: 508 total; 503 residual
```

The estimates from the two model fits are similar. What about the residuals? The *normalized* residuals

$$\mathbf{r} = \hat{\sigma}^{-1}(\hat{\mathbf{V}}^{-1/2})^T(\mathbf{y} - \hat{\mathbf{y}})$$

should be approximately normally distributed as independent $N(0, I)$ random values if the covariance model is correct (Pinheiro and Bates 2000, pg. 239).

```
par(mfrow = c(3,1))
acf(resid(g1), las = 1, main = "residuals")

resid.gls <- resid(g1)

corstr <-  corARMA(c(0.3939, 0.4381), p = 2)
corstr <- Initialize(corstr, data = lap)
CM <- corMatrix(corstr)

## normalized residuals (Pinheiro & Bates pg. 239)
stres <- 1 / g1$sigma * t(solve(chol(CM))) %*% resid.gls
plot(stres, las = 1, pch = 19)
acf(stres, las = 1, main = "normalized residuals")
```

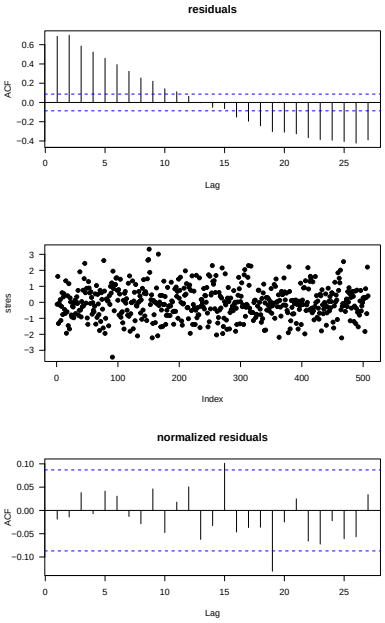


Figure 12: Residuals and normalized residuals for mortality data and AR(2) error model.

4.5 Forecasting from regression with ARMA errors, and from ARMAX

For the ARMAX model, provided we have a prediction for x_{t+1}, the one-step ahead forecast is obtained by substituting the observed values (or forecasts for longer horizons).

$$y_t = \beta_0 + \beta_1 x_t + \phi y_{t-1} + \beta_2 x_{t-1} + e_t$$

To obtain one-step ahead forecasts for the ARMA-error model

$$y_t = \beta_0 + \beta_1 x_t + \phi \eta_{t-1} + e_t$$

we need to forecast both parts, the regression and the ARMA part.

$$\hat{y}_{t+1} = b_0 + b_1 x_{t+1} + \hat{\phi} u_t + 0.$$

For the auto-regressive error terms η_t we substitute observed residuals:

$u_t = y_t - Xb$ or $u_t = \hat{y}_t - Xb$ if y_t is not available.

5 Transfer function models

Transfer function models deal with a combination of lagged response, explanatory and ARMA error terms. These are used in econometrics and engineering. The theory of estimation and forecasting is beyond the scope of this course. Although the above models are special cases, the theory for transfer function models has a strong ARIMA approach and looks quite different to the few simple models we have looked at above. Also note that the approach and terminology used in engineering and econometrics may differ.

6 GARMA

Not all data can be modelled using a Gaussian distribution. For example, time series of counts are very common, e.g. number of malaria or polio cases, number of customers, internet traffic. Variables such as rainfall and streamflow are not normal. All these variables are commonly measured over time, but so far we have not encountered any methods that can deal with non-normal data.

One useful feature of the ARMA error model is that this approach of modelling autocorrelation through the error structure can be extended to generalized models (non-normal data), resulting in Generalized Autoregressive Moving Average (GARMA) models (Benjamin et al., 2003).

In R GARMA models can be fitted using the `garmaFit` from library `gamlss.util`.

The GARMA model is defined as

$$g(\mu_t) = \eta_t = x_t^T\beta + \sum_{k=1}^{p} \phi_k(g(y_{t-k}) - x_{t-k}^T\beta) + \sum_{k=1}^{q} \theta_k(g(y_{t-k}) - \eta_{t-k}). \tag{6}$$

The first part is the usual linear predictor with covariates. The second part contains auto-regressive terms, and the 3rd part moving average terms. To see the difference between the auto-regressive and moving average terms, we can write out the case of a normal linear model with ARMA(1, 1) errors:

$$\begin{aligned} y_t &= x_t^T\beta + \epsilon_t \\ &= x_t^T\beta + \phi\epsilon_{t-1} + \theta e_{t-1} + e_t \end{aligned}$$

The linear predictor part here is $\eta_t = x_t^T\beta + \phi\epsilon_{t-1} + \theta e_{t-1}$.

$y_t - x_t^T\beta = \epsilon_t$

$y_t - \eta_t = e_t$

There are two types of error in the GARMA model:

1. the errors with an ARMA correlation structure, and
2. the independent errors. η_t could be regarded as the fitted value.

The parameter vectors β, ϕ and θ are estimated by maximum likelihood.

The AR terms look very similar to the MA terms, but contain everything not captured by the explanatory variables, whereas in the MA terms the linear predictor accounts for previous errors. Albarracin et al. (2019) have shown that there remains a high degree of correlation between the AR and MA terms, leading to model fitting and interpretation problems and they suggested using either only AR terms or only MA terms.

6.1 Example: Polio cases

We will look at the monthly number of poliomyelitis cases in the US for 1970-1983 (`polio` in `gamlss.data`, this is the data used for the example in `garmaFit`). There are 168 observations. We will add harmonic terms (sine and cosine) to model seasonal changes over the year (our explanatory variables for this example). A negative binomial model is used to account for extra variance (variance in these counts are larger than can be modelled using a Poisson distribution), and two MA terms are included (this is the model fitted by Benjamin et al. 2003).

```
library(gamlss.util)

df <- data.frame(pol = as.numeric(polio), mnth = 0:167)
m02 <- garmaFit(polio ~ cos(2*pi*mnth/12) +
                        sin(2*pi*mnth/12) +
                        cos(4*pi*mnth/12) +
                        sin(4*pi*mnth/12),
                order = c(0, 2), family = NBI, data = df)
summary(m02)
```

`sigma` is the dispersion parameter of the negative binomial distribution. Judging how good a model is, is much more difficult in the Generalized Linear Modelling framework than it is for linear regression models. Visual checks are important, and usually several models are fitted and compared using AIC or a similar criterion. The results from the above fitted model suggest that there is some seasonal pattern in polio cases, and the residuals are uncorrelated. The seasonal effect estimates from a model without autocorrelated errors are similar,

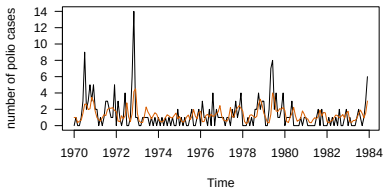


Figure 13: Polio cases with observed counts in black and fitted values from a GARMA(0, 2) model in red.

although with smaller standard errors, and some autocorrelation left in the residuals.

```
m02.est <- coef(m02)

mnth <- 1:24
seas.pred <- exp(m02.est[1] +
                 m02.est[2] * cos(2*pi*mnth/12) +
                 m02.est[3] * sin(2*pi*mnth/12) +
                 m02.est[4] * cos(4*pi*mnth/12) +
                 m02.est[5] * sin(4*pi*mnth/12))
```

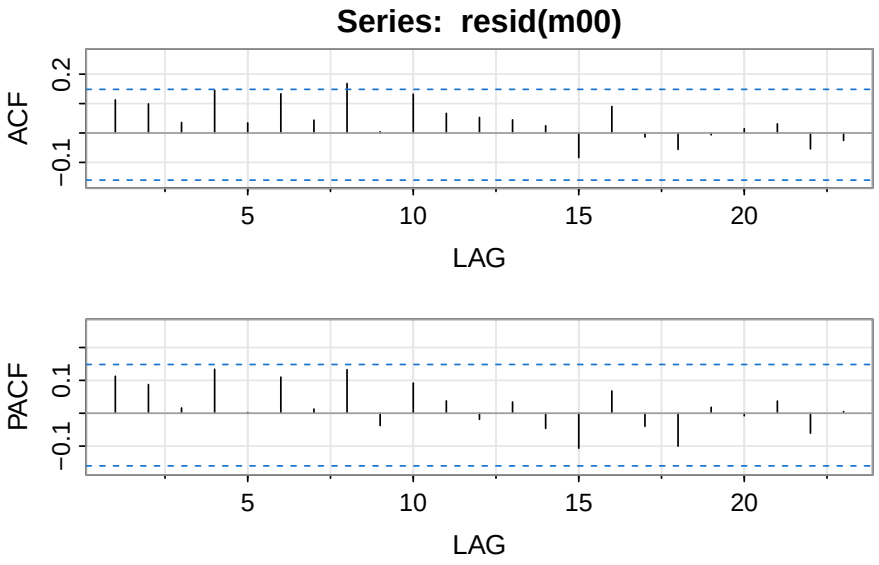
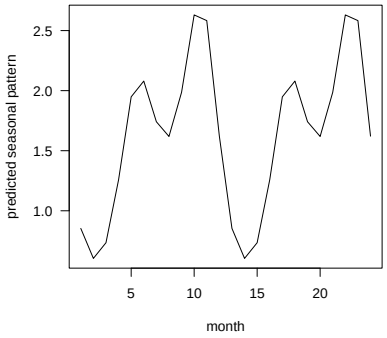


Figure 15



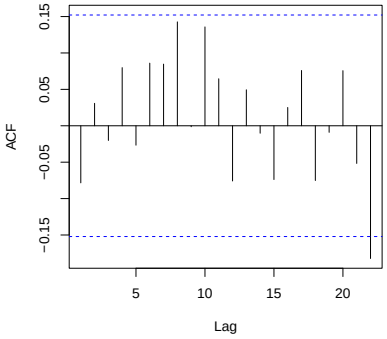


Figure 14: Estimated seasonal pattern and ACF of residuals for polio data from GARMA(0, 2) model.

7 Vector Autoregressive (VAR) Models

VAR models are used for modelling a set of k interrelated time series (multivariate time series). In this approach there is no clear response variable but two or more time series depend on lags of each other.

Suppose we have $k = 2$ time series y_t and z_t. The VAR(1) model is

$$y_t = \alpha_1 + \beta_1 y_{t-1} + \beta_2 z_{t-1} + e_t$$
$$z_t = \alpha_2 + \beta_3 y_{t-1} + \beta_4 z_{t-1} + w_t.$$

All variables from the left-hand-side also appear as lagged variables on the right hand side of each equation. In matrix form:

$$y_t = \alpha + B y_{t-1} + w_t$$

where y_t is a *vector* of observations, B is a $k \times k$ 'transition' matrix, and w_t is assumed to be a multivariate normal white noise process with mean 0 and covariance matrix

$$E(w_t w_t^{'}) = \Sigma.$$

If $E(y_t) = \mu$, then $\alpha = (I - B)\mu$. The VAR(p) model further assumes that all k time series are stationary.

The p-lag vector autoregressive (Var(p)) model is

$$y_t = \alpha + \beta_1 Y_{t-1} + \beta_2 Y_{t-2} + \beta_p Y_{t-p} + e_t$$

β are coefficient matrices, e_t is an unobservable white noise process, with time invariant covariance matrix Σ.

$Cov(e_t, w_s) = \sigma_{ew}$ for $t = s$, 0 otherwise.

e_t are independent over time, but e_t may be correlated with w_t.

7.1 Example: Simulated Data

We can simulate data from the following VAR(1) process:

$$y_t = -0.7 + 0.7 y_{t-1} + 0.2 z_{t-1} + e_t$$
$$z_t = 1.3 + 0.2 y_{t-1} + 0.7 z_{t-1} + w_t$$

$$\left(\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 0.7 & 0.2 \\ 0.2 & 0.7 \end{pmatrix} \right) \begin{pmatrix} 1 \\ 5 \end{pmatrix} = \begin{pmatrix} 0.3 & -0.2 \\ -0.2 & 0.3 \end{pmatrix} \begin{pmatrix} 1 \\ 5 \end{pmatrix} = \begin{pmatrix} 0.3 - 1 \\ -0.2 + 1.5 \end{pmatrix} = \begin{pmatrix} -0.7 \\ 1.3 \end{pmatrix}$$

with $\sigma_{ew} = 0.5, \sigma_e^2 = \sigma_w^2 = 1$.

```
set.seed(44)
B = matrix(c(0.7,0.2,0.2,0.7), 2, 2)
mu.vec = c(1, 5)
alpha = as.vector((diag(2) - B) %*% mu.vec)
Sigma = matrix(c(1, 0.5, 0.5, 1), nrow = 2)

set.seed(301)

k <- 2    # number of time series
p <- 1    # number of lags
N <- 200            # number of observations

library(SimDesign)

series <- matrix(0, k, N + 2*p)
```

```
for (i in (p + 1):(N + 2*p)){
  series[, i] <- alpha + B %*% series[, i-1] +
                 matrix(rmvnorm(1, sigma = Sigma), nrow = 2)
}
series <- ts(t(series[, -(1:p)])) # Convert to time series format
```

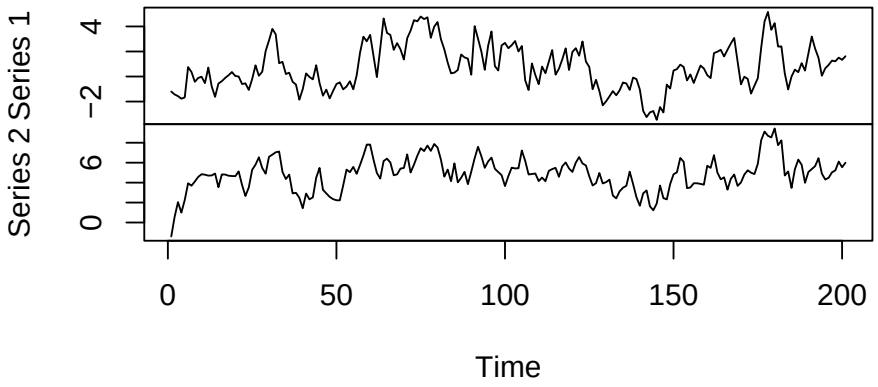


Figure 16: Two time series simulated from a VAR(1) process.

We can fit each equation separately, using least squares (normal linear regression model). Alternatively we can use the `VAR` function from library `vars` (which probably uses the same fitting approach). The covariance for the errors is calculated from the residuals as

$$\frac{1}{N-k}\sum e_t' e_t$$

where k is the number of estimated parameters (3 in each equation = 6).

```
library(dynlm)
head(series)
y <- series[,1]
z <- series[,2]

my <- dynlm(y ~ L(y) + L(z))
summary(my)

mz <- dynlm(z ~ L(y) + L(z))
summary(mz)

res.y <- resid(my)
res.z <- resid(mz)

cov(data.frame(res.y, res.z)) * 199 / 197
```

Conditional on the observed values of the other time series, $Y_i t$ follows a normal linear regression model, and estimation of the parameters in one time series does not depend on information or parameter estimates from the other time series.

```r
library(vars)
m.var <- VAR(series, 1, type = "const")
summary(m.var)
```

The estimates from the two fitting approaches are identical. The estimates for the covariance matrix are based on 197 independent observations per series, hence the degrees of freedom for each series and variance is 197.

7.2 Impulse Response Function

When autoregressive terms are present, it is difficult to interpret the effects of variables on each other. We often would like to know how a shock (sudden spike) in one variable affects the other variable. Because of the autoregressive terms in the model, such shocks may have an effect that lasts for a while. This is what impulse response functions do. One simulates a shock through an impulse of one unit in the error term and calculates the effect of this on the output variables. (Impulse response functions are also used in transfer function models).

In the following code the effect of a disturbance / shock in series 1 on series 2 is estimated.

```r
plot(ir1, xlim = c(0, 25))
```

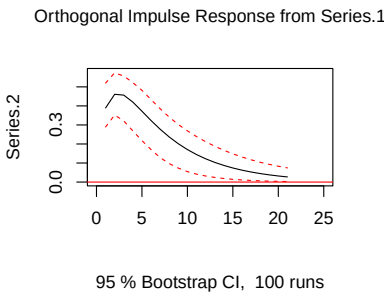


Figure 17: Impulse response function for 2 simulated series from a VAR(1) process. Effect on series 2 after an impulse from series 1 at time zero.

This plot suggests that after a 1 unit increase in series 1 at time 0, series 2 will show an increased response from time 1, which will further increase until 3 time units after the shock and then slowly start to decay towards 0, until about 20 units after the disturbance.

7.3 Adding explanatory variables (exogenous)

The VAR(p) model can be extended to include explanatory variables on the right hand side. These are called exogenous, because they only act as inputs, but are not modelled as part of the system.

$$y_t = \Gamma u_t + \sum_{j=1}^{p} \phi_j x_{t-j} + w_t$$

where Γ is a $p \times r$ parameter matrix. u_t is a vector of explanatory (exogenous) variables. This extension may be referred to as a vector ARX model.

References for these VAR notes: Chatfield (2003) (pg. 246 Section 12.3), Shumway and Stoffer (2011) (Section 5.8 (Multivariate ARMAX Models)).

This introduction just scratches the surface of VAR models. There are many more specialized types of models, and much more theory, but you should now have some idea of what VAR models are and what they can do for you.

8 References

Albarracin, Orlando Yesid Esparza, Airlane Pereira Alencar, and Linda Lee Ho. 2019. "Generalized Autoregressive and Moving Average Models: Multicollinearity, Interpretation and a New Modified Model." *Journal of Statistical Computation and Simulation* 89 (10): 1819–40. https://doi.org/10.1080/00949655.2019.1599892.

Chatfield, C. 2003. *The Analysis of Time Series: An Introduction*. 6th ed. Chapman; Hall/CRC.

Hooten MB, McClintock BT, Johnson DS. 2017. *Animal Movement: Statistical Models for Telemetry Data*. CRC Press.

Hyndman, & Athanasopoulos, R. J. 2021. "Forecasting: Principles and Practice." 3rd ed. OTexts: Melbourne, Australia. OTexts.com/fpp3.

Keele, Luke, and Nathan J. Kelly. 2006. "Dynamic Models for Dynamic Theories: The Ins and Outs of Lagged Dependent Variables." *Political Analysis* 14 (2): 186–205. https://doi.org/10.1093/pan/mpj006.

Mizon, Grayham E. 1995. "A Simple Message for Autocorrelation Correctors: Don't." *Journal of Econometrics* 69 (1): 267–88. https://doi.org/10.1016/0304-4076(94)01671-L.

Pinheiro, J C, and D M Bates. 2000. *Mixed-Effects Models in S and S-PLUS*. Springer.

Rigby, RA, and DM Stasinopoulos. 2005. "Generalized Additive Models for Location, Scale and Shape." *Journal of the Royal Statistical …* 54: 507–54. http://onlinelibrary.wiley.com/doi/10.1111/j.1467-9876.2005.00510.x/full.

Shumway, RH, and DS Stoffer. 2011. *Time Series Analysis and Its Applications: With R Examples*. 3rd ed. http://www.stat.pitt.edu/stoffer/tsa3/tsa3ez.pdf.

The Mathworks Inc. n.d. "Time Series Regression VIII: Lagged Variables and Estimator Bias." https://www.mathworks.com/help/

econ/time-series-regression-viii-lagged-variables-and-estimator-bias.html.

Wilkins, Arjun S. 2018. "To Lag or Not to Lag?: Re-Evaluating the Use of Lagged Dependent Variables in Regression Analysis." *Political Science Research and Methods* 6 (2): 393–411. https://doi.org/10.1017/psrm.2017.4.

- Benjamin M. A., Rigby R. A. and Stasinopoulos D.M. (2003) Generalised Autoregressive Moving Average Models. J. Am. Statist. Ass., 98, 214-223.
- deMeritt J. Time Series Models. https://jdemeritt.weebly.com/uploads/2/2/7/7/22771764/timeseries.pdf
- Hyndman R. The ARIMAX model muddle. Blog post. https://robjhyndman.com/hyndsight/arimax/
- Hyndman & Athanasopoulos. Forecasting: Principles and Practice. Chapters 7, 10. https://otexts.com/fpp3/.
- Pfaff B. 2008. VAR, SVAR and SVEC Models: Implementation Within R Package vars. Journal of Statistical Software 27(4). http://www.jstatsoft.org/v27/i04/.
- Stasinopoulos D. M., Rigby R.A. and Akantziliotou C. (2006) Instructions on how to use the GAMLSS package in R. Accompanying documentation in the current GAMLSS help files, (see also http://www.gamlss.org/).
- Stasinopoulos D. M. Rigby R.A. (2007) Generalized additive models for location scale and shape (GAMLSS) in R. Journal of Statistical Software, Vol. 23, Issue 7, Dec 2007, http://www.jstatsoft.org/v23/i07.